

1. Practical1

Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using `fork()`. 3. Execute the command in the child process using an appropriate `exec()` system call. 4. Allow the parent process to wait for the child using `wait ()`. 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (`uname`, `lscpu`, `lsblk`, `ps`, `top`), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main(){
    char command[100];
    pid_t pid;
    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);
    command[strcspn(command, "\n")] = '\0';
    pid = fork();
    if (pid < 0){
        perror("Fork failed");
        return 1;
    }else if (pid == 0){
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
        execlp(command, command, NULL);
        perror("Execution failed");
        exit(1);
    }else{
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        wait(NULL);
        printf("\nChild process completed.\n");
    }
    return 0;
}
```

Output:

```
sai776421@Dra: ~
sai776421@Dra:~$ nano Week1.c
sai776421@Dra:~$ gcc Week1.c
sai776421@Dra:~$ ./a.out
Enter a Linux command: ls

Parent Process
Parent PID: 561

Child PID: 562
Child Process
Child PID: 562
Parent PID: 561
FCFS.c    SJF.c    a.out    program2.c  program4.c  program6.c  program8.c
Robin.c   Week1.c  program1  program3.c  program5.c  program7.c  week2.c

Child process completed.
```

2. Practical2

Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char *argv[]){
    int source_fd, dest_fd;
    ssize_t bytes_read, bytes_written;
    char buffer[1024];

    if (argc != 3){
        printf("Usage: %s <source_file> <destination_file>\n", argv[0]);
        return 1;
    }

    source_fd = open(argv[1], O_RDONLY);
    if (source_fd < 0){
        perror("Error opening source file");
        return 1;
    }

    dest_fd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if(dest_fd < 0){
        perror("Error opening destination file");
        close(source_fd);
        return 1;
    }

    while ((bytes_read = read(source_fd, buffer, sizeof(buffer))) > 0){
        bytes_written = write(dest_fd, buffer, bytes_read);

        if (bytes_written != bytes_read){
            perror("Write error");
            close(source_fd);
            close(dest_fd);
            return 1;
        }
    }if(bytes_read < 0){
        perror("Read error");
    }
    close(source_fd);
    close(dest_fd);
    printf("File copied successfully.\n");
    return 0;
}

```

Output:

```

sai776421@Dra:~$ nano Week2.c
sai776421@Dra:~$ nano Practical2.c
sai776421@Dra:~$ gcc Practical2.c -o Practical2
sai776421@Dra:~$ ./Practical2
File copied Successfully
sai776421@Dra:~$ cat sample.txt
Hello World
sai776421@Dra:~$ cat copy.txt
Hello World

```

ii) Strace cat sample.txt

Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as `ps`, `top`, and `/proc`. Document the observations.

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(){
    pid_t pid;
    pid = fork();

    if(pid < 0){
        printf("Fork Failed\n");
        return 1;
    }else if(pid == 0){
        printf("\n----- CHILD PROCESS ----- \n");
        printf("Child PID   : %d\n", getpid());
        printf("Child PPID  : %d\n", getppid());

        printf("Child is running...\n");
        sleep(10);

        printf("Child process terminating...\n");
        exit(0);
    }else{
        printf("\n----- PARENT PROCESS ----- \n");
        printf("Parent PID  : %d\n", getpid());
        printf("Child PID   : %d\n", pid);

        printf("Parent waiting for child...\n");
        wait(NULL);

        printf("Child terminated.\n");
        printf("Parent process terminating...\n");
    }
    return 0;
}
```

Output

```
sai776421@Dra:~$ nano Week3.c
sai776421@Dra:~$ gcc Week3.c
sai776421@Dra:~$ ./a.out

----- PARENT PROCESS -----

----- CHILD PROCESS -----
Parent PID : 1531
Child PID  : 1532
Parent waiting for child...
Child PID  : 1532
Child PPID : 1531
Child is running...
Child process terminating...
Child terminated.
Parent process terminating...
```

4. Practical4

Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(){
    int i;
    pid_t pid;

    for(i = 1; i <= 3; i++){
        pid = fork();

        if(pid == 0){
            printf("Child %d: PID = %d, PPID = %d\n",
                i, getpid(), getppid());
            sleep(i * 2);
            printf("Child %d completed.\n", i);
            exit(0);
        }
    }

    printf("\nParent PID = %d\n", getpid());
    for(i = 1; i <= 3; i++){
        pid_t child = wait(NULL);
        printf("Parent collected child with PID %d using wait()\n",
            child);
    }
    printf("All child processes terminated.\n");
    return 0;
}

```

Output

```

sai776421@Dra:~$ nano Week4.c
sai776421@Dra:~$ gcc Week4.c
sai776421@Dra:~$ ./a.out
Child 1: PID = 1599, PPID = 1598
Child 2: PID = 1600, PPID = 1598

Parent PID = 1598
Child 3: PID = 1601, PPID = 1598
Child 1 completed.
Parent collected child with PID 1599 using wait()
Child 2 completed.
Parent collected child with PID 1600 using wait()
Child 3 completed.
Parent collected child with PID 1601 using wait()
All child processes terminated.

```

Zombie.c

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(){
    pid_t pid1, pid2;
    pid1 = fork();

    if(pid1 == 0){
        printf("Child 1 PID = %d\n", getpid());
        sleep(5);
        printf("Child 1 completed\n");
        exit(0);
    }

    pid2 = fork();

    if(pid2 == 0){
        printf("Child 2 PID = %d\n", getpid());
        sleep(2);
        printf("Child 2 completed\n");
        exit(0);
    }

    waitpid(pid2, NULL, 0);
    printf("Parent collected Child 2 using waitpid()\n");
    waitpid(pid1, NULL, 0);
    printf("Parent collected Child 1 using waitpid()\n");
    printf("All child processes terminated.\n");
    return 0;
}

```

Output

```

sai776421@Dra:~$ nano Zombie.c
sai776421@Dra:~$ gcc Zombie.c -o Zombie
sai776421@Dra:~$ ./Zombie
Child 1 PID = 1639
Child 2 PID = 1640
Child 2 completed
Parent collected Child 2 using waitpid()
Child 1 completed
Parent collected Child 1 using waitpid()
All child processes terminated.

```

5. Practical5

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>
#include <time.h>

int main(){
    int fd[2];
    pid_t pid;
    char buffer[100];

    if(pipe(fd) == -1){
        perror("Pipe");
        exit(1);
    }
    pid = fork();

    if(pid < 0){
        perror("Fork");
        exit(1);
    }
    if(pid > 0){
        close(fd[0]);
        clock_t start = clock();

        char data[] = "Hello from Producer (Parent Process)";
        write(fd[1], data, strlen(data) + 1);

        clock_t end = clock();
        double time_taken =
            ((double)(end - start)) / CLOCKS_PER_SEC;

        printf("Producer sent data.\n");
        printf("Communication Time = %lf seconds\n", time_taken);

        close(fd[1]);
        wait(NULL);
    }
    else{
        close(fd[1]);
        read(fd[0], buffer, sizeof(buffer));
        printf("Consumer received: %s\n", buffer);
        close(fd[0]);
    }
    return 0;
}
```

Output

```

sai776421@Dra:~$ nano Week5.c
sai776421@Dra:~$ gcc Week5.c
sai776421@Dra:~$ ./a.out
Producer sent data.
Consumer received: Hello from Producer (Parent Process)
Communication Time = 0.000071 seconds

```

ii) producer consumer communication using anonymous pipe

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(){
    int fd[2];
    if(pipe(fd) == -1){
        perror("Pipe Failed");
        exit(1);
    }

    pid_t pid1 = fork();
    if(pid1 == 0){

        close(fd[0]);
        dup2(fd[1], STDOUT_FILENO);

        close(fd[1]);
        execlp("ls", "ls", "-l", NULL);

        perror("exec ls failed");
        exit(1);
    }

    pid_t pid2 = fork();
    if(pid2 == 0){

        close(fd[1]);
        dup2(fd[0], STDIN_FILENO);

        close(fd[0]);
        execlp("grep", "grep", ".c", NULL);
        perror("exec grep failed");
        exit(1);
    }
    close(fd[0]);
    close(fd[1]);

    wait(NULL);
    wait(NULL);
    return 0;
}

```

Output

```

sai776421@Dra:~$ nano Wprogram5.c
sai776421@Dra:~$ gcc Wprogram5.c -o Wprogram5
sai776421@Dra:~$ ./Wprogram5
-rw-r--r-- 1 sai776421 sai776421 388 Sep 1 03:30 FCFS.c
-rwxr-xr-x 1 sai776421 sai776421 16176 Sep 14 13:43 Practical2
-rw-r--r-- 1 sai776421 sai776421 609 Sep 14 13:30 Practical2.c
-rw-r--r-- 1 sai776421 sai776421 610 Sep 1 03:36 Robin.c
-rw-r--r-- 1 sai776421 sai776421 496 Sep 1 03:33 SJF.c
-rw-r--r-- 1 sai776421 sai776421 859 Sep 14 10:53 Week1.c
-rw-r--r-- 1 sai776421 sai776421 1114 Sep 14 13:29 Week2.c
-rw-r--r-- 1 sai776421 sai776421 849 Sep 14 13:53 Week3.c
-rw-r--r-- 1 sai776421 sai776421 698 Sep 14 13:59 Week4.c
-rw-r--r-- 1 sai776421 sai776421 953 Sep 14 14:09 Week5.c
-rw-r--r-- 1 sai776421 sai776421 726 Sep 14 15:21 Wprogram5.c
-rw-r--r-- 1 sai776421 sai776421 712 Sep 14 14:02 Zombie.c
-rw-r--r-- 1 sai776421 sai776421 12 Sep 14 13:44 copy.txt
-rw-r--r-- 1 sai776421 sai776421 258 Aug 4 03:01 program2.c
-rw-r--r-- 1 sai776421 sai776421 278 Aug 4 03:01 program3.c
-rw-r--r-- 1 sai776421 sai776421 164 Aug 4 03:03 program4.c
-rw-r--r-- 1 sai776421 sai776421 215 Aug 4 03:04 program5.c
-rw-r--r-- 1 sai776421 sai776421 304 Aug 4 03:05 program6.c
-rw-r--r-- 1 sai776421 sai776421 174 Aug 4 03:06 program7.c
-rw-r--r-- 1 sai776421 sai776421 846 Aug 4 03:12 program8.c
-rw-r--r-- 1 sai776421 sai776421 2050 Sep 11 15:49 week2.c

```

6. Practical6 Server

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```

#include<stdio.h>
#include<fcntl.h>
#include<unistd.h>
#include<sys/stat.h>
#include<string.h>
int main(){
int fd;
char message[100];
mkfifo("/tmp/myfifo",0666);
fd = open("/tmp/myfifo",O_RDONLY);
read(fd,message,sizeof(message));
printf("Server received the message: %s\n",message);
close(fd);
return 0;
}

```

Client

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>

int main() {
    int fd;
    char message1[] = "Hello world";

    fd = open("/tmp/myfifo", O_WRONLY);

    write(fd, message1, strlen(message1) + 1);

    printf("Client sent the message: %s\n", message1);

    close(fd);
    return 0;
}
```

Output

```
karamsettisaicharan@SaiCharan:~$ ./practical6_1
Client sent the message: Hello world
karamsettisaicharan@SaiCharan:~$ |
```

```
karamsettisaicharan@SaiCharan:~$ ./practical6
Server received the message: Hello world
```

ii) SIGNALS

```
GNU nano 8.7.1          Signal.c
#include <stdio.h>
#include <signal.h>
void handler(int sig)
{
    if(sig == SIGINT)
        printf("SIGINT received\n");
    if(sig == SIGTERM)
        printf("SIGTERM received\n");
    if(sig == SIGUSR1)
        printf("SIGUSR1 received\n");
}
int main()
{
    signal(SIGINT, handler);
    signal(SIGTERM, handler);
    signal(SIGUSR1, handler);
    printf("Signal handling program\n");
    raise(SIGINT);
    raise(SIGTERM);
    raise(SIGUSR1);
    return 0;
}
```

Output

```
sai776421@Dra:~$ nano Week6.2.c
sai776421@Dra:~$ gcc Week6.2.c
sai776421@Dra:~$ ./a.out
Signal handling program
SIGINT received
SIGTERM received
SIGUSR1 received
```